

Maintenance Plan

Software Support Strategy and Operational Rhythm

1. Maintenance Categories

Maintenance activities are systematically categorized to guide prioritization:

- **Corrective Maintenance:** Focuses on resolving critical defects, including authentication failures, bugs in core job posting/application flows, and data integrity issues.
- **Adaptive Maintenance:** Addresses modifications needed due to changes in the operating environment, such as the migration to the Supabase/PostgreSQL backend or updates to hosting infrastructure.
- **Perfective Maintenance:** Concentrates on product enhancement, including improving the User Experience (UX), performance tuning, refining input validation, and optimizing search or pagination features.

2. Roles and Operational Rhythm

Lightweight Roles & Responsibilities

Role	Primary Responsibilities
Maintainer	Triage incoming issues, assign owners, conduct code reviews, and manage releases.
Developer	Implement fixes and new features, and design necessary tests and smoke plans.
QA (If Available)	Reproduce reported issues and validate fixes in the dedicated staging environment.

Operating Rhythm

- **Triage:** Conducted weekly to assess new issues.
- **Release Window:** Biweekly or Monthly release window (for patch or minor versions) depending on the size of the backlog and project deadlines.

3. Issue Intake and Triage

Group-8

A structured process ensures all reported issues and requests are handled efficiently:

1. **Submission:** All reported issues (bugs or features) must be submitted via the designated tracker, including:
 - Clear steps to reproduce the issue.
 - Expected vs. actual results.
 - Relevant screenshots or logs.
2. **Weekly Triage:** Issues are classified by severity and type:
 - **Severity:** Blocker, Major, or Minor.
 - **Type:** Bug, Feature, or Chore.
3. **Assignment:** A primary owner is assigned, and a target release (patch or minor) is scheduled.

Severity & Target Response Example

Severity	Target Acknowledgment Time	Target Release
Blocker (Outage, Data Loss)	Within 4 business hours	Hotfix patch (Immediate)
Major (Core Flow Degraded)	Within 1 business day	Next scheduled patch release
Minor (Cosmetic, Edge Case)	Scheduled in the next minor release	

4. Workflow: Bug Fixing (Corrective Maintenance)

1. **Reproduction:** Reproduce the issue in the development environment. Document clear steps or add a failing test case.
2. **Implementation:** Create the fix in a dedicated, short-lived branch. Prioritize security best practices (e.g., using prepared statements for MySQL prototype or robust input validation).
3. **Smoke Tests:** Run a minimum set of smoke tests to verify core functions: registration, login, job posting, application submission, and dashboard navigation.
4. **Peer Review:** Conduct a rigorous code review, specifically checking for: security vulnerabilities (SQL injection, session handling), potential regressions, and adherence to established coding patterns.
5. **Deployment Verification:** Deploy the fix to a test/staging environment to verify the specific correction and re-test all critical application paths.

6. **Release:** Tag the release, update the changelog, and close the issue ticket.

5. Workflow: Features and Enhancements (Perfective/Adaptive)

1. **Request Grooming:** Define precise acceptance criteria for the feature and assess its impact on the data model, noting any necessary Supabase schema changes.
2. **Design Update:** Plan the revised API/data flow, confirm correct role handling logic (Company vs. Job Seeker), and outline required database migrations.
3. **Implementation:** Build the feature incrementally, using small, testable units. Continue to enforce security standards and robust validation.
4. **Review and Testing:** Perform unit and smoke tests covering authentication, posting, applying, and listing functionalities.
5. **Release:** Deploy the feature as part of a minor version update and ensure all related configuration changes are fully documented.

Minimum Regression & Smoke Tests

- **Authentication:** Register, login (both employer and seeker roles), and logout.
- **Employer:** Post job, view postings, view applications.
- **Job Seeker:** View job listings, view job detail, and apply for a job.
- **Listing:** Verify the homepage correctly renders the latest jobs and associated company names.

6. Data Protection and Continuity

Backups and Security

- **Supabase:** Automated daily backups must be enabled. API key exposure must be restricted, and keys should be rotated immediately if any potential leak is detected.
- **Prototype MySQL (If Active):** Manual SQL database dumps must be exported before and after any major changes.

Service Continuity & Rollback

- **Supabase:** If deployment fails, roll back the application to the previous tagged version. Utilize Supabase's point-in-time recovery features for database restoration if necessary.
- **Prototype:** Restore the MySQL SQL dump and revert the codebase to the previous application tag.

Post-Release Monitoring

- **Metrics:** Continuously check Supabase metrics (error rates, latency) and hosting provider logs.
- **User Feedback:** Actively track user-reported issues and prioritize corrective maintenance based on impact.

7. Documentation

Group-8

All project documentation must be maintained and kept current with every release:

- README.md
- The Deployment Plan
- Supabase configuration notes
- A dedicated docs/ folder for technical guides.